

The Islamia University of Bahawalpur
Department of Software Engineering



Software Requirements Specification (SRS)

Project Title: AI-Based Quranic Verse Search Engine

Developed by: Muhammad Ali

Roll No: S22BSEEN1M01052

Submission Date: 21-05-2025

Session Fall 2022 – 2026

Supervisor

Dr. Muhammad Nauman

Lecturer Department of Software Engineering

Bachelor of Science in Software Engineering

Table of Contents

1. Introduction	
1.1 Purpose-----	5
1.2 Scope-----	5
1.3 Definitions, Acronyms, and Abbreviations-----	6
1.4 References-----	7
1.5 Overview-----	7
2. Overall Description	
2.1 Product Perspective-----	7
2.2 Product Functions-----	8
2.3 User Classes and Characteristics-----	8
2.4 Operating Environment-----	9
2.5 Design and Implementation Constraints-----	9
2.6 User Documentation-----	10
2.7 Assumptions and Dependencies-----	10
3. Specific Requirements	
3.1 Functional Requirements-----	10
3.2 Non-Functional Requirements-----	12
3.3 Interface Requirements-----	13
3.4. System Features-----	14
4. System Architecture	
4.1 Data Flow Diagram-----	15
4.2 System Architecture Diagram-----	15
5. Use Cases	
5.1 Use Case Diagrams-----	16
5.2 Detailed Use Case Descriptions-----	16
6. Data Dictionary-----	17

7. References-----	18
--------------------	----

Revision History

Name	Date	Reason for changes	Version
Initial Draft	21-05-2025	Created Initial version of SRS	1.0

Application Evaluation History

Comments (by committee) *include the ones given at scope time both in doc and presentation	Action Taken

Supervised by

Dr Muhammad Nauman

Signature_____

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the complete requirements for the development of the **AI-Based Quranic Verse Search Engine**. This project aims to bridge the gap between modern technology and religious accessibility by creating an intelligent search system that allows users to search for Quranic verses based on natural language input, such as a topic or a question. The software will analyze the semantic meaning of the query and retrieve relevant verses from the Quran using Natural Language Processing (NLP) and Machine Learning techniques.

The document describes the software's objectives, functionality, intended users, system environment, constraints, and performance expectations. It also outlines the functional and non-functional requirements of the system, design details, and external interface specifications. This document is intended to guide the development process, ensure all stakeholders are aligned, and serve as a reference for validation and future maintenance.

The software's primary goal is to assist users in discovering verses of the Quran in a meaningful way without requiring knowledge of Arabic or exact verse references. The engine will support **semantic search** and **multilingual display** enhancing the spiritual and educational experience for users.

1.2 Scope

The **AI-Based Quranic Verse Search Engine** is a web-based application that utilizes Natural Language Processing to provide semantic search functionality over the Quran. Users can input a phrase, topic, or even a full question (e.g., "What does the Quran say about forgiveness?"), and the system will intelligently match the query with relevant verses.

AI-Based Quranic Verse Search Engine

Key features of the system include:

- **Natural Language Input:** Users can ask questions or enter keywords without needing exact phrasing.
- **Semantic Search:** The engine understands meaning and context using TF-IDF or pre-trained BERT models (e.g., SBERT).
- **Multilingual Output:** Each matched verse will be displayed in Arabic, with English and Urdu translations.
- **Offline Capability:** The software will run locally and not require an internet connection for core functionality.
- **User-Friendly Interface:** Built using Streamlit or Flask for a clean and accessible user experience.
- **Expandable Modules:** Future additions like audio playback, tafsir (commentary), and multiple translation options are planned.

The system does **not require a centralized database**; it will use locally stored JSON datasets for speed and privacy. This makes the system lightweight and suitable for offline environments like Islamic schools or personal study.

1.3 Definitions, Acronyms, and Abbreviations

- **SRS:** Software Requirements Specification
- **AI:** Artificial Intelligence
- **NLP:** Natural Language Processing
- **TF-IDF:** Term Frequency-Inverse Document Frequency
- **UI:** User Interface
- **JSON:** JavaScript Object Notation
- **Streamlit:** A Python library for building web applications

1.4 References

This document and the software design are inspired and supported by several open-source initiatives and academic literature:

1. Tanzil.net - Verified Quran Dataset <https://tanzil.net/>
2. Quran.com API (for optional extensions)
3. Sentence-Transformers Documentation <https://www.sbert.net/>
4. Scikit-learn Documentation <https://scikit-learn.org/>
5. Streamlit Documentation <https://docs.streamlit.io/>
6. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications

These references provide datasets, tools, and guidelines to ensure the accuracy, efficiency, and usability of the developed system.

1.5 Overview

This document is organized into sections that cover the complete software development life cycle. From the general description of the system to specific features, non-functional requirements, and appendices, this SRS is structured to be a definitive reference for both development and validation teams.

2. Overall Description

2.1 Product Perspective

This product is a new and standalone web-based system. It is not dependent on any existing system but will use standard data formats (JSON) and third-party libraries (NLTK, scikit-learn, sentence-transformers) to perform core functionalities.

2.2 Product Functions

Functionality	Description
User Input Handling	Accepts keyword-based or full-sentence queries in English.
Semantic Search	Utilizes NLP models to identify and retrieve Quranic verses semantically related to the input.
Verse Retrieval	Displays verses in Arabic along with English and Urdu translations.
Surah & Ayah Metadata	Each result includes Surah name and Ayah number.
Result Ranking	Matches are ranked based on semantic similarity scores.
Language Toggle	Users can choose between English, Urdu, or both translations.
Theming and Accessibility	Offers a minimalistic, accessible user interface (light and dark modes).
Offline Functionality	Core features will function without internet access after installation.
Future Expansion Hooks	Easily extendable to support Tafsir, audio recitation, or mobile deployment.

2.3 User Classes and Characteristics

User Type	Description
General Public	Users interested in learning from the Quran
Students	Islamic studies, research purposes
Scholars	Researchers looking for quick reference
Developers	Future maintainers or contributors

2.4 Operating Environment

Software Requirements:

- **Operating System:** Windows 10+, Ubuntu/Linux, macOS
- **Programming Language:** Python 3.8 or later

- **Libraries/Frameworks:**

- Streamlit (Web UI)
- scikit-learn (TF-IDF vectorizer)
- Sentence-Transformers (SBERT for semantic search)
- pandas (Data manipulation)
- NumPy (Numerical computing)
- json (For data loading)

Deployment Options:

- Local execution (via Python script or Streamlit app)
- Optional: Hosted on platforms like **Streamlit Cloud**, **Heroku**, or **Render** for live access

2.5 Design and Implementation Constraints

Constraint	Description
Offline-first	System must function without requiring internet access after initial setup.
Privacy & Ethics	User queries are not stored or transmitted. All processing is local to preserve privacy.
Language Support	Arabic (source), English and Urdu (translations) must be displayed correctly. Unicode and right-to-left formatting must be properly handled.
Performance	Semantic models must be lightweight enough to run on standard consumer laptops. Use of distilled or optimized models is preferred.
Open-source datasets only	All Quranic texts and translations used must be verifiable and freely distributable under open licenses.

2.6 User Documentation

- README with setup and usage instructions
- Developer guide (optional)

- In-app help tooltips

2.7 Assumptions and Dependencies

- Users are familiar with basic Islamic terminology
 - Internet is available for optional features only (audio, updates)
 - All dependencies are available through PyPI
-

3. Specific Requirements

3.1 Functional Requirements

Functional requirements describe the interactions between the system and its users, covering inputs, outputs, and the processing that occurs.

3.1.1 User Interface

- **FR1.1:** The system shall provide a search bar where users can type natural language queries in English.
- **FR1.2:** The system shall display search results in the form of Quranic verses that are semantically similar to the input query.
- **FR1.3:** The system shall display each matched verse in:
 - Arabic (original)
 - English translation
 - Urdu translation
- **FR1.4:** The system shall display Surah name and verse number with each result.
- **FR1.5:** The system shall display the similarity score (optional) to indicate how closely each verse matches the query.

3.1.2 Query Processing

- **FR2.1:** The system shall accept a natural language query as user input.
- **FR2.2:** The system shall use a vectorization technique (e.g., TF-IDF or Sentence-BERT) to convert both the query and verses into embeddings.
- **FR2.3:** The system shall compare the query vector with stored verse vectors using a similarity measure (e.g., cosine similarity).
- **FR2.4:** The system shall return the top N (default = 5) most relevant verses based on semantic similarity.

3.1.3 Multilingual Support

- **FR3.1:** The system shall allow the user to toggle between English and Urdu translations.
- **FR3.2:** The system shall correctly render right-to-left Arabic and Urdu text.
- **FR3.3:** The system shall use verified translations from open-source Quran datasets (e.g., Sahih International, Urdu Jalandhry).

3.1.4 Data Handling

- **FR4.1:** The system shall load Quranic text and translations from local JSON or CSV files.
- **FR4.2:** The system shall pre-process the verse dataset to create vector embeddings during initialization (only once or when required).
- **FR4.3:** The system shall store processed embeddings for faster search on subsequent runs.

3.1.5 Additional Features (Optional/Planned)

- **FR5.1:** The system shall include a button for playing audio recitation of a verse (future extension).
- **FR5.2:** The system shall allow the user to copy or export a verse and its translation.

- **FR5.3:** The system shall support verse bookmarking or tagging for user reference (future scope).

3.2 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system, such as performance, usability, scalability, and reliability.

3.2.1 Performance Requirements

- **NFR1.1:** The system shall return search results within 2 seconds for queries on machines with minimum recommended hardware.
- **NFR1.2:** The system shall use optimized models (e.g., distilbert or paraphrase-MiniLM) to minimize resource usage and response time.

3.2.2 Usability Requirements

- **NFR2.1:** The system shall provide a clean, responsive, and accessible user interface.
- **NFR2.2:** The system shall be usable by users without prior training or technical background.
- **NFR2.3:** The system shall follow UI/UX best practices, including clear font, readable text, and intuitive layout.

3.2.3 Reliability & Availability

- **NFR3.1:** The system shall be available offline once installed and initialized.
- **NFR3.2:** The system shall not require an internet connection to perform core semantic search functions.
- **NFR3.3:** The system shall operate without crashing or data corruption for at least 95% of queries.

3.2.4 Maintainability

- **NFR4.1:** The system codebase shall be modular and follow clean code principles.

- **NFR4.2:** The system shall use configuration files or constants for model selection, path handling, and data loading.
- **NFR4.3:** The system documentation shall include setup, usage, and developer extension guides.

3.2.5 Portability

- **NFR5.1:** The system shall be compatible with Windows, Linux, and macOS platforms.

3.3 Interface Requirements

3.3.1 User Interfaces

The user interface shall be web-based (using **Streamlit** or **Flask**). It will consist of:

- Search bar input for the query.
- Dropdown or toggle for translation language selection.
- Scrollable section displaying results (verse + translation + metadata).
- Optional: Icons or buttons for bookmarking or playing recitation.

3.3.2 Hardware Interfaces

- No specific hardware interfaces required beyond standard computing hardware (PC or laptop with Python support).

3.3.3 Software Interfaces

Component	Interface
NLP Engine	Sentence Transformers or Scikit-learn for vectorization
Data Files	JSON or CSV input files containing Quranic text and translations
Web Interface	Streamlit or Flask rendering engine

3.3.4 Communications Interfaces

- Not applicable for the basic offline version.
 - Optional for hosted deployment: HTTP/HTTPS protocols via browser interface.
-

3.4. System Features

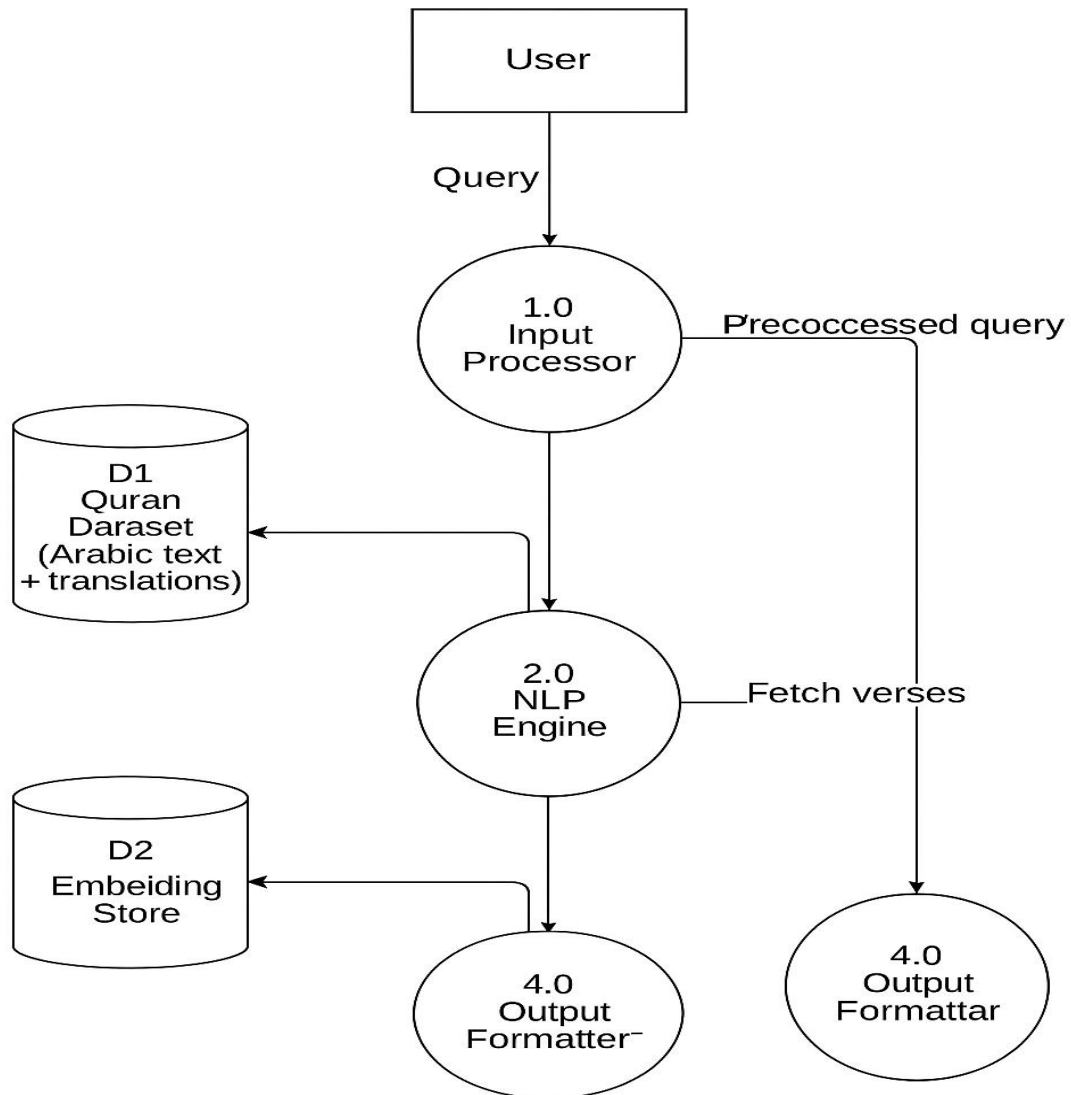
Feature	Type	Mandatory/Optional
Natural Language Search	Functional	Mandatory
Semantic Matching	Functional	Mandatory
Arabic + Translation Display	Functional	Mandatory
Multilingual Support (Urdu/English)	Functional	Mandatory
Responsive Web UI	Non-Functional	Mandatory
Audio Playback	Functional	Optional
Export/Copy Verse	Functional	Optional
Offline Capability	Non-Functional	Mandatory
Expandable with Tafsir	Functional	Optional

4. System Architecture

This section describes the high-level architecture of the system, including how data flows through the components and how those components interact with each other. The purpose of this section is to give a visual and conceptual overview of the internal workings of the system and to guide the development and integration of individual modules.

4.1 Data Flow Diagram

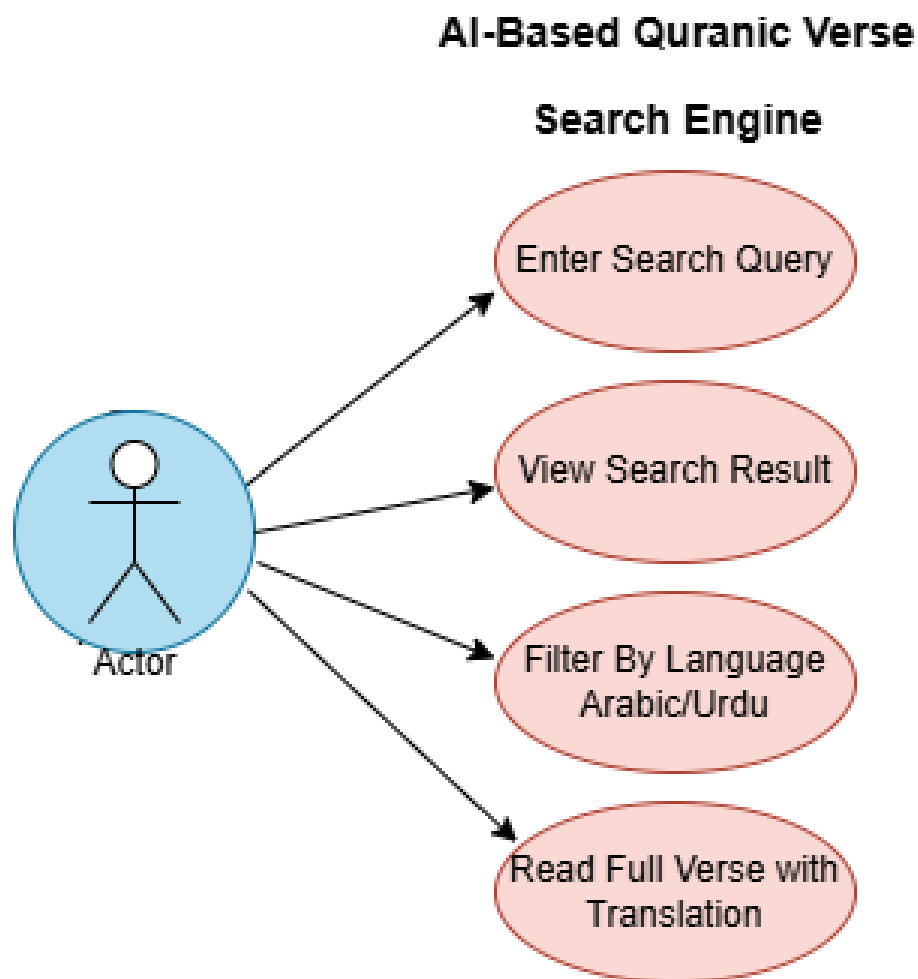
The Data Flow Diagram represents the logical flow of information through the system, from user input to system output. It helps visualize how data is processed, stored, and presented.



AI-Based Quranic Search Engine
Data Flow Diagram (Level 1)

5. Use Cases

5.1 Use Case Diagram



5.2 Detailed Use Cases

Use Case: Search Verse

- Actor: User
- Precondition: System is running

- Postcondition: Relevant verses displayed
- Main Flow:
 1. User enters query
 2. System processes input
 3. Results ranked and displayed

6. Data Dictionary

Field	Type	Description
surah	String	Name of Surah
ayah	Int	Verse number
arabic	String	Original Arabic text
english	String	English translation
urdu	String	Urdu translation

7. UML Diagram

7.1 Class Diagram

Classes:

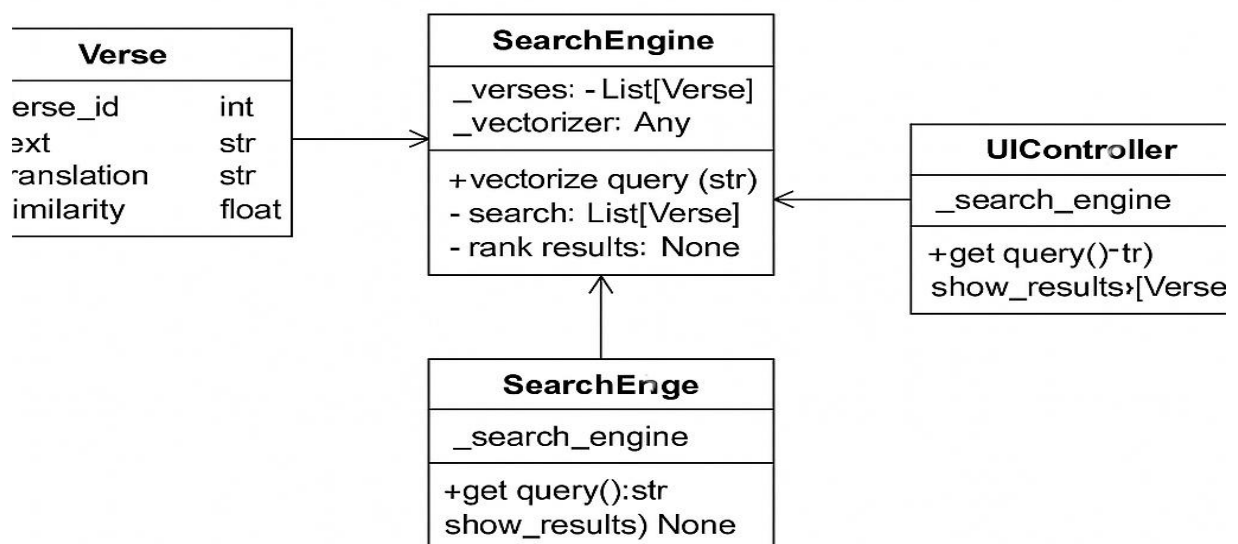
- **Verse**
 - verse_id: int
 - text: str
 - translation: str
 - similarity: float

- **SearchEngine**

- `_verses: List[Verse]`
- `_vectorizer: Any`
- `+vectorize_query(query: str) -> Vector`
- `+search(query: str) -> List[Verse]`
- `-rank_results() -> List[Verse]`

- **UIController**

- `_search_engine: SearchEngine`
- `+get_query() -> str`
- `+show_results(verses: List[Verse]) -> None`

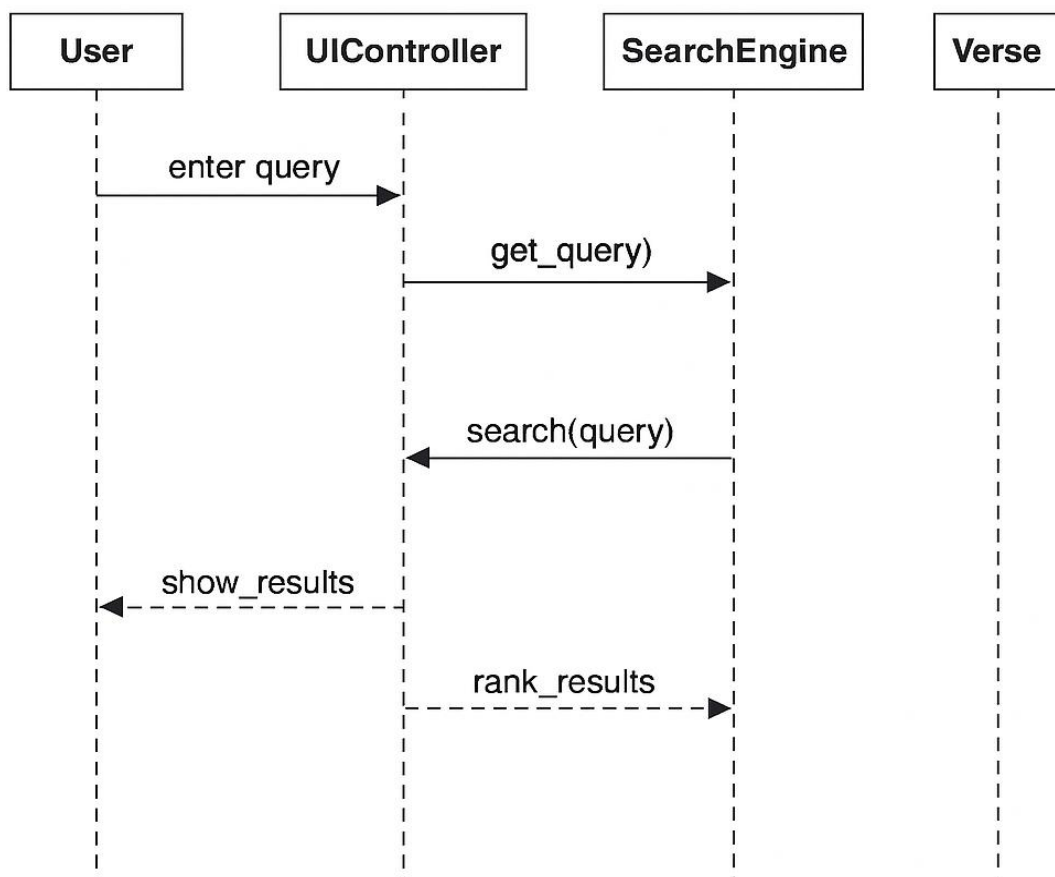


7.2 Sequence Diagram

Process: User submits a query and receives search results.

Lifelines:

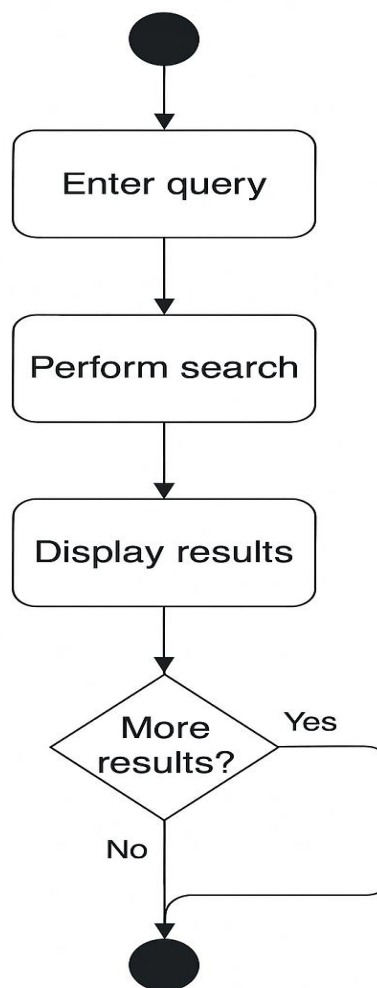
- User
- UIController
- SearchEngine
- Verse



7.3 Activity Diagram

Flow:

1. **Start**
2. **Enter query**
3. **Perform search**
4. **Display results**
5. **Decision: More results?**
 - If **Yes** → Back to "Enter query"
 - If **No** → **End**



9. References

1. The Noble Qur'an (Arabic Text + Translations)

- Quranic Arabic Corpus. <http://corpus.quran.com/>
- Tanzil Quran Text. <https://tanzil.net/download/>
- Quran JSON API and datasets. <https://github.com/fawazahmed0/quran-api>

2. Urdu and English Quran Translations

- Urdu: Translation by Maulana Fateh Muhammad Jalandhry (جالندھری)
- English: Sahih International Translation
- Quran.com GitHub. <https://github.com/quran/quran.com-frontend>

3. Natural Language Processing (NLP)

- Reimers, Nils, and Iryna Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. arXiv preprint arXiv:1908.10084, 2019.

- HuggingFace Transformers. <https://huggingface.co/transformers/>
- Scikit-learn Library. <https://scikit-learn.org/stable/>
- Sentence Transformers Documentation. <https://www.sbert.net/>

4. Python & Libraries

- Python Official Docs. <https://docs.python.org/3/>
- Pandas Library. <https://pandas.pydata.org/>
- NumPy. <https://numpy.org/>

5. Web Frameworks

- Flask Framework Documentation. <https://flask.palletsprojects.com/>
- Streamlit Documentation. <https://docs.streamlit.io/>

6. System Design and SRS Guidelines

- IEEE 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
- Sommerville, Ian. *Software Engineering*. 10th Edition. Pearson Education.
- Pressman, Roger S. *Software Engineering: A Practitioner's Approach*. 8th Edition.

7. Testing Resources

- pytest documentation. <https://docs.pytest.org/>
- Unit Testing in Python. <https://realpython.com/python-testing/>

8. Diagrams and UML Standards

- UML 2.5 Specification. Object Management Group (OMG).
<https://www.omg.org/spec/UML/2.5/>
- Lucidchart. <https://www.lucidchart.com> (for creating DFDs and UML diagrams)

9. Similar Research Projects and Repositories

- Quran Search Project. <https://github.com/> (search for: "Quran semantic search" or "Quran NLP")
 - Islamic NLP Research Papers on arXiv. <https://arxiv.org/>
-

* Additional Notes:

- For academic purposes, cite translations using author and title if needed (e.g., *Sahih International Translation of the Quran*).
-